Logfire Python SDK — Full Security & Compliance Audit

Date: 2026-03-31
Project: `logfire` v4.31.0 + `logfire-api` v4.31.0
Repository: `https://github.com/pydantic/logfire`
Audit Framework: Microsoft Security Development Lifecycle (SDL)
Methodology: Architecture Discovery → Threat Modeling → Parallel Domain Audits
Static Analysis: CodeQL skipped per user request — all findings are from manual review

---

Table of Contents

1. Executive Summary
2. Architecture Overview
3. Review Methodology
4. Threat Model Summary
5. Findings by Domain
   - 5.1 Security
   - 5.2 Privacy
   - 5.3 Responsible AI
   - 5.4 Supply Chain
   - 5.5 Digital Safety
   - 5.6 Accessibility

6. Cross-Cutting Issues
7. Static Analysis Summary
8. Severity Distribution
9. SDL Phase Distribution
10. Prioritised Recommendations
11. Detailed Findings Reference

---

1. Executive Summary

The Logfire Python SDK is a well-engineered OpenTelemetry-based observability library. It ships with thoughtful defaults in many areas: scrubbing is on by default, TLS is enforced throughout, OIDC Trusted Publishing is used for PyPI releases, and the lockfile contains SHA256 hashes for every dependency. However, this audit identified **86 raw findings** across six domains (4 Critical, 23 High, 33 Medium, 26 Low) before deduplication. After accounting for 6 cross-cutting findings that appear in multiple domain reports, there are approximately **76 unique findings**.

Severity Summary (raw, by domain)

Domain	Critical	High	Medium	Low	Total
Security	0	4	6	5	15
Privacy	1	7	6	5	19
Responsible AI	2	4	6	4	16
Supply Chain	0	3	7	5	15
Digital Safety	0	3	4	2	9

Domain	Critical	High	Medium	Low	Total
Accessibility	1	2	4	5	12
Total	**4**	**23**	**33**	**26**	**86**

Top 5 Issues Requiring Immediate Action

1. 🟤 **LLM scrubbing bypass** (`scrubbing.py:200-202`) — All PII redaction is silently skipped for OpenAI and Anthropic spans. User-supplied personal data in prompts/responses is transmitted verbatim to the Logfire backend. *(Security H-2, Privacy Critical, Digital Safety DS-01, RAI C-01)*

2. 🟤 **No opt-out for LLM content capture** — `instrument_openai()` and `instrument_anthropic()` unconditionally capture full prompts and responses with no `capture_content=False` equivalent. This blocks GDPR data minimisation compliance for production AI applications. *(Privacy Critical/High, RAI C-02)*

3. 🟤 **Claude AI GitHub Action with unrestricted trigger + write permissions** (`.github/workflows/claude.yml`) — Any GitHub user can trigger an AI agent with `contents: write` and `id-token: write` via an issue comment, creating a prompt-injection and supply-chain attack vector into the PyPI release pipeline. *(Security H-3, Supply Chain H-02)*

4. 🟤 **All GitHub Actions pinned to mutable version tags** — No workflow pins actions to commit SHAs. The PyPI release workflow uses `pypa/gh-action-pypi-publish@release/v1` (a mutable branch reference), making a supply chain attack on PyPI a realistic scenario. *(Security H-4, Supply Chain H-01, H-03)*

5. 🟤 **Credential files written without restricted permissions** — `~/.logfire/default.toml` and `.logfire/logfire_credentials.json` are written without `chmod 600`; they are world-readable on systems with a standard umask. *(Security M-1)*

---

2. Architecture Overview

Full document: `audit/ARCHITECTURE.md`

Attribute	Details
Type	Open-source Python SDK (library) + CLI + documentation
Language	Python 3.9–3.14 (primary)
Core Protocol	OpenTelemetry (traces, metrics, logs) → OTLP/Protobuf over HTTPS
Backend	Logfire SaaS (`logfire-us.pydantic.dev` / `logfire-eu.pydantic.dev`) — closed source, out of audit scope
Sub-packages	`logfire` (main SDK), `logfire-api` (no-op shim for library authors)
Integrations	40+ auto-instrumentation targets: FastAPI, Django, Flask, SQLAlchemy, OpenAI, Anthropic, Pydantic AI, httpx, asyncpg, psycopg, redis, celery, and many more
Credential storage	`~/.logfire/default.toml` (user tokens), `.logfire/logfire_credentials.json` (project write tokens)
Disk persistence	`DiskRetryer` writes failed OTLP payloads to OS temp dir for retry
Distribution	PyPI via GitHub Actions + OIDC Trusted Publishing; uv workspace
Build	hatchling; `uv.lock` with SHA256 hashes

---

3. Review Methodology

Phase	Activity	Output
Phase 0A	Architecture discovery (codebase mapping)	`audit/ARCHITECTURE.md`
Phase 0B	CodeQL static analysis	**Skipped per user request**
Phase 1	Threat modeling (STRIDE + DREAD)	`audit/THREAT_MODEL.md`
Phase 2	Parallel domain audits (6 agents)	`audit/security.md`, `audit/privacy.md`, `audit/responsible-ai.md`, `audit/supply-chain.md`, `audit/digital-safety.md`, `audit/accessibility.md`
Phase 3	Aggregation and deduplication	This document

Languages/frameworks detected: Python (OpenTelemetry SDK, Pydantic, Rich, httpx), GitHub Actions (CI/CD), MkDocs (documentation)

---

4. Threat Model Summary

Full document: `audit/THREAT_MODEL.md`

The threat model identified **20 threats** with DREAD scoring, covering 12 components and 8 trust boundaries.

Threat Register Summary

Severity	Count	Key Threats
Critical	1	TM-04: LLM prompt/response capture without consent
High	4	TM-01: Base URL hijack (SSRF/token theft); TM-03: Scrubbing bypass; TM-05: SQL parameter PII; TM-18: Supply chain compromise
Medium	15	Token exposure in logs, disk retry file exposure, trace injection, remote variable tampering, Pydantic plugin implicit activation, OAuth device flow abuse, read token leakage

STRIDE Distribution

STRIDE Category	Dominant Threats
Spoofing	Base URL redirect; fake Logfire backend
Tampering	AST rewriting hook; DiskRetryer payload modification
Repudiation	No immutable audit log in SDK
Information Disclosure	LLM prompt capture; credential file exposure; disk retry plaintext; URL query PII
Denial of Service	Unbounded span generation; 512 MB DiskRetryer accumulation
Elevation of Privilege	GitHub Actions write-permission AI agent; supply chain via unpinned actions

Top 3 Mitigations from Threat Model

1. **M-02** — Default `capture_input=False` / `capture_output=False` for all LLM instrumentation
2. **M-01** — Domain allowlist / SSRF guard on `LOGFIRE_BASE_URL`

3. **M-03** — Extend scrubbing patterns; apply SQL parameter value scrubbing

---

5. Findings by Domain

5.1 Security

Full report: `audit/security.md`
Summary: 0 Critical · 4 High · 6 Medium · 5 Low

High Findings

ID	Title	File	SDL Phase
SEC-H1	SSRF / Write-Token Theft via `LOGFIRE_BASE_URL` — no domain allowlist; env var can redirect all OTLP exports (with `Authorization` header) to an attacker server	`config_params.py:106`, `config.py:1126`	Design
SEC-H2	Scrubbing entirely bypassed for OpenAI/Anthropic spans — `scrub_span()` early-returns for `logfire.openai` / `logfire.anthropic` scopes	`scrubbing.py:200-205`	Implementation
SEC-H3	Claude AI GitHub Action — `contents: write` + `id-token: write` granted to any GitHub user's issue comment; prompt-injection + supply-chain risk	`.github/workflows/claude.yml:16-57`	Release

ID	Title	File	SDL Phase
SEC-H4	GitHub Actions pinned to floating tags, not commit SHAs — compromised action repo could inject malicious code into PyPI release pipeline	`main.yml`, `claude.yml`	Release

Medium Findings

ID	Title	File
SEC-M1	Credential files written without `chmod 600` — world-readable on umask 022	`config.py:1840`, `auth.py:207-214`
SEC-M2	`exception.message` and `exception.stacktrace` exempt from scrubbing — DB passwords, connection strings in exceptions exported unredacted	`scrubbing.py:127-128`
SEC-M3	LLM message content keys globally exempt from scrubbing — `gen_ai.input.messages`, `pydantic_ai.all_messages` in `SAFE_KEYS`	`scrubbing.py:147-157`
SEC-M4	`LOGFIRE_CREDENTIALS_DIR` not validated for symlink attacks	`utils.py:215-222`
SEC-M5	OAuth device flow `poll_for_token()` has no wall-clock timeout — can block CI indefinitely	`auth.py:261-275`
SEC-M6	`OTEL_RESOURCE_ATTRIBUTES` allows arbitrary key injection without sanitisation	`config.py:971-975`

Low Findings

ID	Title
SEC-L1	DiskRetryer temp files have no explicit permissions on Windows
SEC-L2	Credential files lack integrity check (no HMAC/signature)
SEC-L3	`logfire run` logs the full command string, potentially exposing CLI args with secrets
SEC-L4	CLI activity log `~/.logfire/log.txt` created without `chmod 600`
SEC-L5	Write token not validated for format before use in `DiskRetryer` retry payloads

---

5.2 Privacy

Full report: `audit/privacy.md`
Summary: 1 Critical · 7 High · 6 Medium · 5 Low

Data Inventory Summary

The SDK can capture and transmit to the Logfire backend: LLM prompts/responses (HBI), SQL query text (MBI–HBI), HTTP URLs and query strings (MBI), HTTP headers incl. Authorization/Cookie when enabled (HBI), function arguments when auto-traced (MBI–HBI), exception messages and stack traces (MBI), Pydantic model field data (MBI–HBI), MCP tool arguments/results (MBI–HBI), write tokens (HBI), OAuth user tokens (HBI), process metadata (LBI), and unencrypted retry payloads on disk (mirrors all of the above).

Critical Finding

ID	Title	File	SDL Phase

ID	Title	File	SDL Phase
PRI-C1	LLM spans bypass scrubbing entirely; no content opt-out for OpenAI/Anthropic — full prompts, responses, and all user PII transmitted verbatim to backend	`scrubbing.py:199-205`, `main.py:1224-1425`	Implementation / Design

High Findings

ID	Title	File
PRI-H1	`db.statement` in SAFE_KEYS — SQL queries never scrubbed, including dynamically constructed queries containing user data	`scrubbing.py:138`
PRI-H2	HTTP URLs and query strings in SAFE_KEYS — never scrubbed despite frequently containing tokens, session IDs, and user-identifying parameters	`scrubbing.py:133-146`
PRI-H3	Exception messages in SAFE_KEYS — user-supplied data in validation errors, DB constraint errors exported unredacted	`scrubbing.py:128-130`
PRI-H4	Unencrypted telemetry persisted to disk in retry queue — up to 512 MB of raw Protobuf in OS temp dir with no encryption at rest	`exporters/otlp.py:105-217`
PRI-H5	No opt-out for LLM content capture in OpenAI/Anthropic instrumentation — inconsistent with Pydantic AI and Google GenAI which have opt-out	`main.py:1224-1314, 1328-1425`
PRI-H6	HTTP header capture can expose Authorization and Cookie values when `capture_headers=True` — not clearly warned in docs	`utils.py:383-387`, `integrations/fastapi.py:78`
PRI-H7	`record='all'` on Pydantic plugin captures entire model input/output including PII fields without field-level redaction	`integrations/pydantic.py`

Medium Findings

ID	Title
PRI-M1	No privacy notice or data inventory shown before first telemetry transmission
PRI-M2	`collect_system_info` transmits installed package list without documented disclosure
PRI-M3	No data retention controls in the SDK; no mechanism to trigger deletion at the backend
PRI-M4	`scrubbing=False` disables all redaction silently — no SecurityWarning emitted
PRI-M5	OTel baggage forwarded from inbound requests may contain PII
PRI-M6	No documentation of GDPR data processing role (controller vs. processor distinction for SDK users)

Low Findings

ID	Title
PRI-L1	`logfire.configure()` docstring does not list data categories collected
PRI-L2	No mechanism for right-to-erasure request at SDK level
PRI-L3	DiskRetryer does not delete retry files on successful export acknowledgement with zero retry remaining
PRI-L4	`logfire read-tokens create` output is not masked in terminal output
PRI-L5	Third-party OTel instrumentation packages (google-genai, openinference) may capture data per their own policies

---

5.3 Responsible AI

Full report: `audit/responsible-ai.md`
Summary: 2 Critical · 4 High · 6 Medium · 4 Low

The SDK instruments **10 external LLM providers/frameworks**: OpenAI, Anthropic (including Bedrock), Google Generative AI, LiteLLM, DSPy, OpenAI Agents SDK, Pydantic AI, Claude Agent SDK, MCP, and Logfire's own eval and `logfire prompt` CLI features.

Critical Findings

ID	Title	File	SDL Phase
RAI-C1	Scrubbing pipeline completely disabled for OpenAI/Anthropic spans — negates all RAI privacy protections	`scrubbing.py:200-202`	Implementation
RAI-C2	No opt-out mechanism for LLM prompt/response capture in OpenAI and Anthropic instrumentation	`main.py:~1224, ~1328`	Design

High Findings

ID	Title	File
RAI-H1	LLM message content attributes (`gen_ai.input.messages` , etc.) in `SAFE_KEYS` — secondary bypass layer	`scrubbing.py:111-160`
RAI-H2	Binary/image/audio content captured without restrictions — `BlobPart` captures base64 biometric data with no size limits or content allowlists	`integrations/llm_providers/types.py`
RAI-H3	No content safety filtering on AI outputs — LLM responses stored verbatim with no toxicity detection	Pipeline-wide
RAI-H4	Unpinned AI instrumentation dependencies — `openinference-instrumentation-litellm >= 0` accepts any future version	`pyproject.toml:84-85`

Medium Findings

ID	Title
RAI-M1	Inconsistent LLM content controls — opt-out exists for Pydantic AI and Google GenAI but not OpenAI/Anthropic
RAI-M2	MCP tool call arguments/results captured without filtering — prompt injection storage risk
RAI-M3	No model card or system card for `logfire prompt` CLI or AI eval features
RAI-M4	No fairness evaluation support in the evals framework
RAI-M5	No human-in-the-loop, escalation mechanism, or kill switch for AI decisions
RAI-M6	`logfire prompt` CLI serves AI-generated content with no model disclosure

Low Findings

ID	Title
RAI-L1	EU AI Act risk classification absent from documentation
RAI-L2	No `system_fingerprint` capture for model version reproducibility
RAI-L3	No anomaly detection hooks in telemetry pipeline
RAI-L4	GDPR Art. 22 automated-decision disclosure gap

5.4 Supply Chain

Full report: `audit/supply-chain.md`
Summary: 0 Critical · 3 High · 7 Medium · 5 Low
SLSA Level: 2 of 4

High Findings

ID	Title	File	SDL Phase
SC-H1	Branch-pinned actions in release critical path — `pypa/gh-action-pypi-publish@release/v1` and `re-actors/alls-green@release/v1` use mutable branch refs; force-push would silently backdoor next PyPI release	`main.yml`	Release
SC-H2	Claude AI workflow grants write permissions to external trigger — any GitHub user can invoke AI agent with `contents: write` + `id-token: write` via issue comment	`claude.yml`	Design
SC-H3	All GitHub Actions unpinned to commit SHAs (tag-only) — 11+ action refs across 5 workflow files; `astral-sh/setup-uv` installs uv binary itself	All workflows	Verification

Medium Findings

ID	Title	File
SC-M1	Build backend (`hatchling`) completely unpinned — first code executed in source builds	`pyproject.toml:2`
SC-M2	`pydantic-docs` VCS dependency without commit SHA — fetches GitHub HEAD; runs with `ALGOLIA_WRITE_API_KEY` in env	`pyproject.toml:221`
SC-M3	Docs build bypasses lockfile via `--upgrade` and private registry — `mkdocs-material` pulled without hash verification	`main.yml:46-48`

ID	Title	File
SC-M4	No SLSA provenance attestations or artifact signing	Release pipeline
SC-M5	Weekly CI installs Pydantic from Git HEAD without commit SHA — runs with `SLACK_WEBHOOK_URL` in env	`weekly_deps_test.yml:75`
SC-M6	No automated dependency vulnerability scanning — no Dependabot, no `pip-audit` in CI	CI configuration
SC-M7	`openinference` optional deps effectively unbounded (`>= 0`) — LLM data access with no version floor	`pyproject.toml:84-85`

Low Findings

ID	Title
SC-L1	No SBOM generated in release pipeline
SC-L2	Third-party Depot runners used for CI — security posture unverified
SC-L3	`allow-direct-references = true` permits arbitrary VCS deps in workspace
SC-L4	`protobuf >= 4.23.4` lower bound includes CVE-2024-7254 range — raise to `>= 4.25.4`
SC-L5	`opentelemetry-instrumentation >= 0.41b0` broad lower bound

5.5 Digital Safety

Full report: `audit/digital-safety.md`
Summary: 0 Critical · 3 High · 4 Medium · 2 Low

Note: Classical digital safety concerns (compulsive design, photosensitivity, age-gating) are not applicable to a developer SDK. All findings are SDK-level data safety issues.

High Findings

ID	Title	File
DS-H1	LLM input/output exempt from scrubbing by design — all AI prompt/response data transmitted verbatim	`scrubbing.py:148-160, 200-202`
DS-H2	SQL statement attribute (`db.statement`) exempt from scrubbing	`scrubbing.py:137`
DS-H3	`scrubbing=False` silently disables all PII redaction with no `SecurityWarning`	`config.py` (NoopScrubber path)

Medium Findings

ID	Title
DS-M1	SDK sends host application metadata (PID, Python version, git hash, UUID) without prominent disclosure
DS-M2	Pydantic plugin activates implicitly on `import pydantic` before `logfire.configure()` is called
DS-M3	No rate-limiting or volume controls on span generation — `install_auto_tracing()` can generate millions of spans/second

ID	Title
DS-M4	DiskRetryer accumulates up to 512 MB with no cleanup on shutdown

Low Findings

ID	Title
DS-L1	No pre-send data inventory or first-run disclosure mechanism
DS-L2	Console exporter prints HTTP URLs and SQL statements in shared CI environments

---

5.6 Accessibility

Full report: `audit/accessibility.md`
Summary: 1 Critical · 2 High · 4 Medium · 5 Low
WCAG Conformance: Non-conformant at Level A, AA, and AAA

Scope: CLI tool, console exporter (Rich output), MkDocs documentation. Python SDK API and OTLP exporter are not applicable.

Critical Finding

ID	Title	File	WCAG	SDL Phase
ACC-C1	Log severity conveyed by color alone — no `[ERROR]` / `[WARN]` / `[INFO]` text labels; 8% of male developers are red-green color blind	`exporters/console.py:207-216`	1.4.1 (Level A)	Implementation

High Findings

ID	Title	File	WCAG
ACC-H1	Primary brand color `#E520E9` fails WCAG AA contrast (3.67:1) on white in light mode docs	`docs/extra/tweaks.css:2-6`	1.4.3 (Level AA)
ACC-H2	Search results list has `role="presentation"` — removes screen-reader list semantics and result count	`docs/overrides/partials/search.html:27`	4.1.2 (Level A)

Medium Findings

ID	Title	File	WCAG
ACC-M1	Unicode-only status indicators (`✓` , `⚠` , `□`) in CLI instrumentation summary	`cli/run.py:227-231`	1.1.1 (Level A)
ACC-M2	Search result links have `tabindex="-1"` , blocking Tab-key navigation	`docs/javascripts/algolia-search.js:95`	2.1.1 (Level A)
ACC-M3	Screenshot images have minimal/non-descriptive alt text across docs	`docs/**/*.md`	1.1.1 (Level A)
ACC-M4	`NO_COLOR` environment variable not documented or tested	`config_params.py` , `docs/reference/cli.md`	1.4.1 principle

Low Findings

ID	Title	WCAG
ACC-L1	CSS transition missing `prefers-reduced-motion` fallback	2.3.3 (Level AAA)
ACC-L2	Rich panel box-drawing characters announced verbatim by screen readers	1.3.1 (Level A)
ACC-L3	Silent browser-open failure provides no distinct error message	3.3.1 (Level A)
ACC-L4	Auth error messages do not specify the exact command to run	3.3.3 (Level AA)
ACC-L5	Search toggle icon label has no accessible name	1.1.1, 4.1.2 (Level A)

---

6. Cross-Cutting Issues

Several findings appear independently across multiple domain audits and represent the highest-leverage remediation targets.

CC-1 — LLM Scrubbing Bypass (CRITICAL)

Appears in: Security (SEC-H2), Privacy (PRI-C1), Digital Safety (DS-H1), Responsible AI (RAI-C1)

The `scrub_span()` early-return for `logfire.openai` and `logfire.anthropic` scopes at `scrubbing.py:200-202` is the single most impactful finding in this audit. It silently negates the SDK's primary privacy protection mechanism for the most sensitive data the SDK captures. This one code block affects GDPR compliance, Microsoft RAI Standard alignment, and general data security simultaneously.

Fix: Remove the scope-level bypass. Add content-specific scrubbing for LLM attributes, and introduce `capture_input` / `capture_output` parameters on `instrument_openai()` and `instrument_anthropic()`.

CC-2 — GitHub Actions Supply Chain Risk (HIGH)

Appears in: Security (SEC-H3, SEC-H4), Supply Chain (SC-H1, SC-H2, SC-H3)

No GitHub Actions are pinned to commit SHAs. The PyPI release path uses a mutable branch reference. The Claude AI workflow grants write and id-token permissions to any public GitHub user.

Fix: Use Ratchet to bulk-pin all actions to SHAs; restrict Claude workflow trigger to repo members; add Dependabot for GitHub Actions.

CC-3 — SAFE_KEYS Exemptions (HIGH)

Appears in: Security (SEC-M2, SEC-M3), Privacy (PRI-H1, PRI-H2, PRI-H3), Digital Safety (DS-H2)

`db.statement`, `http.url`, `url.query`, `exception.message`, `exception.stacktrace`, and the `gen_ai.*` message keys are all in `SAFE_KEYS` and are never scrubbed. These are precisely the attributes most likely to contain PII in production applications.

Fix: Remove PII-prone keys from SAFE_KEYS. Apply query-parameter-aware scrubbing to URL values. Apply value-level pattern scanning to exception messages.

CC-4 — Disk Retry Privacy (HIGH)

Appears in: Privacy (PRI-H4), Digital Safety (DS-M4), Security (SEC-L1)

Unencrypted Protobuf payloads (mirroring all captured telemetry including LLM prompts) persist in the OS temp directory indefinitely with no encryption at rest and no cleanup on shutdown.

Fix: Add an `atexit` cleanup hook; consider per-session ephemeral encryption; enforce time-based retention limits.

CC-5 — `scrubbing=False` Silent Disablement (HIGH)

Appears in: Security (SEC-M2 context), Privacy (PRI-M4), Digital Safety (DS-H3)

Disabling scrubbing produces no warning of any kind. A debug configuration that reaches production silently transmits all PII.

Fix: Emit `warnings.warn("Logfire scrubbing is disabled. All span attributes will be transmitted without PII redaction.", SecurityWarning)` at configure time and on first export.

CC-6 — Inconsistent LLM Content Defaults (HIGH)

Appears in: Privacy (PRI-H5), Responsible AI (RAI-C2, RAI-M1)

Google GenAI is opt-in for content capture; Pydantic AI has `include_content=False`; OpenAI and Anthropic are always-on with no opt-out. This inconsistency makes it impossible to safely instrument multi-provider applications.

Fix: Standardise on explicit `capture_input` / `capture_output` parameters across all LLM instrumentation. Consider defaulting to `False` (privacy-by-default per GDPR Art. 25).

---

7. Static Analysis Summary

CodeQL static analysis was **skipped per user request**. No SARIF file was generated or consumed in this audit. All findings are from manual code review.

Recommendation: Install the CodeQL CLI and run:

```
codeql database create codeql-db --language=python --source-root=.
codeql database analyze codeql-db --format=sarif-latest --output=codeql-results.sarif -- codeql/python-queries
```

This would provide additional coverage for injection patterns, unsafe deserialization, and path traversal vulnerabilities not captured by manual review.

---

8. Severity Distribution

Severity	Count	% of Total
Critical	4	5%
High	23	27%
Medium	33	38%
Low	26	30%
Total	**86**	100%

Note: 6 cross-cutting findings are counted in each domain they appear in. Unique finding count is approximately 76.

---

9. SDL Phase Distribution

SDL Phase	Finding Count	Top Issues
Design	18	SSRF on base URL; LLM content capture by default; no opt-out for LLM data; MCP capture without filtering
Implementation	42	Scrubbing bypasses; credential file permissions; SAFE_KEYS exemptions; accessibility color/role failures
Requirements	8	No privacy notice; no data retention controls; no RAI documentation; EU AI Act gap

SDL Phase	Finding Count	Top Issues
Verification	10	Actions not SHA-pinned; no dependency scanning; no SBOM; no `pip-audit` in CI
Release	6	Branch-pinned PyPI action; Claude workflow permissions; no SLSA provenance
Response	2	No anomaly detection hooks; no incident response plan for telemetry breach

---

10. Prioritised Recommendations

Immediate (Critical / High — address within 1 sprint)

#	Action	Effort	Impact	Fixes
1	Remove scrubbing bypass for `logfire.openai` / `logfire.anthropic` scopes	Low	Critical	CC-1, SEC-H2, PRI-C1, DS-H1, RAI-C1
2	Add `capture_input` / `capture_output` parameters to `instrument_openai()` and `instrument_anthropic()`	Medium	Critical	CC-6, PRI-H5, RAI-C2
3	Pin all GitHub Actions to commit SHAs (use Ratchet)	Low	High	CC-2, SEC-H4, SC-H1, SC-H3
4	Restrict Claude workflow trigger to repo maintainers; remove `id-token: write`	Low	High	SEC-H3, SC-H2
5	Add domain allowlist / SSRF guard on `LOGFIRE_BASE_URL`	Low	High	SEC-H1

#	Action	Effort	Impact	Fixes
6	Apply `chmod 600` to all credential files	Low	Medium	SEC-M1
7	Emit `SecurityWarning` when `scrubbing=False`	Low	High	CC-5, PRI-M4, DS-H3

Short-term (Medium — address within next quarter)

#	Action	Effort	Impact	Fixes
8	Remove `db.statement`, `url.query`, `exception.message` from SAFE_KEYS; apply targeted scrubbing	Medium	High	CC-3, PRI-H1–H3, DS-H2, SEC-M2
9	Add `atexit` cleanup hook for DiskRetryer; add time-based retention limit	Low	Medium	CC-4, PRI-H4, DS-M4
10	Add `capture_content` opt-out for `instrument_openai_agents()` and `instrument_mcp()`	Medium	Medium	RAI-M2, PRI-M5 context
11	Add Dependabot for pip and GitHub Actions	Low	Medium	SC-M6
12	Pin `hatchling` build backend with version range	Low	Medium	SC-M1
13	Add wall-clock timeout to `poll_for_token()` in OAuth device flow	Low	Medium	SEC-M5
14	Fix console exporter color-only severity — add `[ERROR]` / `[WARN]` / `[INFO]` text labels	Low	Medium	ACC-C1
15	Fix docs light-mode primary color contrast (`#E520E9` → `#9900A0`)	Low	Medium	ACC-H1

Longer-term (Low / Compliance — address within 6 months)

#	Action	Effort	Fixes
16	Add `slsa-framework/slsa-github-generator` to release workflow; generate SBOM	Medium	SC-M4, SC-L1
17	Encrypt DiskRetryer payloads at rest using per-session ephemeral key	High	PRI-H4
18	Add `pip-audit` (or `uv audit`) to CI lint job	Low	SC-M6
19	Publish EU AI Act and NIST AI RMF compliance documentation	Medium	RAI-L1
20	Fix docs search accessibility (`role="presentation"`, `tabindex="-1"`, icon labels)	Low	ACC-H2, ACC-M2, ACC-L5
21	Add `prefers-reduced-motion` CSS and descriptive screenshot alt text	Low	ACC-L1, ACC-M3
22	Pin `pydantic-docs` to a commit SHA	Low	SC-M2
23	Add version floors to `openinference-instrumentation-*` optional deps	Low	SC-M7, RAI-H4
24	Raise `protobuf` lower bound to `>= 4.25.4` to clear CVE-2024-7254 range	Low	SC-L4

---

11. Detailed Findings Reference

All sub-agent reports are available in the `audit/` directory:

Report	Domain	File
Architecture Discovery	Architecture	`audit/ARCHITECTURE.md`
Threat Model	Threat Modeling	`audit/THREAT_MODEL.md`

Report	Domain	File
Security Audit	Security	`audit/security.md`
Privacy Audit	Privacy	`audit/privacy.md`
Responsible AI Audit	Responsible AI	`audit/responsible-ai.md`
Supply Chain Audit	Supply Chain	`audit/supply-chain.md`
Digital Safety Audit	Digital Safety	`audit/digital-safety.md`
Accessibility Audit	Accessibility	`audit/accessibility.md`

Audit completed: 2026-03-31. All findings are from manual review; CodeQL static analysis was not run.